



FACULDADE DE TECNOLOGIA SENAI GASPAR RICARDO JÚNIOR

DESENVOLVIMENTO DE UM DASHBOARD HMI PARA MONITORAMENTO DE SENSORES IoT COM NODE.JS, NODE-RED E NEXT.JS

Development of an HMI Dashboard for IoT Sensor Monitoring Using Node.js, Node-RED and Next.js

João Pedro Americo Matias ¹

RESUMO

Este artigo apresenta o desenvolvimento de um sistema de monitoramento IoT composto por uma API RESTful desenvolvida em Node.js, uma interface de dashboard HMI construída com Next.js e React, e um fluxo de simulação de sensores implementado no Node-RED. O sistema permite o monitoramento em tempo real de três sensores, exibindo valores de temperatura, pressão e umidade por meio de gauges circulares interativos, além de indicadores LED para sensores de presença e trava de segurança. A comunicação entre os componentes é realizada por meio de requisições HTTP, seguindo a arquitetura REST. Os resultados demonstram a viabilidade da solução para aplicações de automação industrial, residencial e agrícola.

Palavras-chave: IoT; Node.js; Next.js; Node-RED; Dashboard; HMI; API REST; Sensores.

Keywords: IoT; Node.js; Next.js; Node-RED; Dashboard; HMI; REST API; Sensors.

¹Estudante do curso de Análise e Desenvolvimento de Sistemas — Turma Toyota-3. Faculdade de Tecnologia SENAI Gaspar Ricardo Júnior, Sorocaba — SP, 1º Semestre de 2026. E-mail: joaopedro-matias2007@gmail.com

1 Introdução

A Internet das Coisas (IoT) tem se consolidado como uma das tecnologias mais relevantes para automação e monitoramento em diferentes setores. A capacidade de conectar dispositivos físicos à internet e visualizar seus dados em tempo real representa um avanço significativo para aplicações industriais, residenciais e agrícolas.

Neste contexto, as interfaces HMI (*Human Machine Interface*) desempenham papel central, pois traduzem dados brutos de sensores em informações visuais compreensíveis para operadores e engenheiros. A combinação de ferramentas como Node.js, Node-RED e Next.js permite construir sistemas IoT completos, do backend ao frontend, de forma acessível e escalável.

Este trabalho tem como objetivo apresentar o desenvolvimento de um sistema integrado de monitoramento IoT, composto por uma API RESTful, um fluxo de simulação de sensores via Node-RED e um dashboard HMI com atualização em tempo real, demonstrando a viabilidade técnica da arquitetura proposta.

2 Revisão de Literatura

Segundo Aguiar (2023), a Internet das Coisas pode ser descrita como uma rede de objetos físicos conectados à internet, capazes de interagir por meio de sensores e softwares. O autor destaca que a utilização de APIs RESTful é fundamental para conectar o mundo físico ao digital, permitindo que dispositivos troquem dados de forma eficiente e escalável.

De acordo com Rodrigues (2024), a expansão de funcionalidades em sistemas IoT depende diretamente da integração com APIs HTTP, que possibilitam maior flexibilidade e controle remoto dos dispositivos. A autora enfatiza que o uso de arquiteturas REST garante interoperabilidade entre diferentes plataformas e tecnologias, tornando os sistemas mais adaptáveis às necessidades dos usuários.

O Node-RED é uma ferramenta de programação visual baseada em fluxos desenvolvida pela IBM, amplamente utilizada para integração de dispositivos IoT. Sua arquitetura orientada a mensagens permite conectar sensores, APIs e serviços web de forma simples e visual (IBM, 2023).

Interfaces HMI modernas, segundo Lima (2022), devem priorizar a clareza visual e a atualização em tempo real, características fundamentais para que operadores tomem decisões rápidas com base nos dados exibidos. O uso de gauges circulares e indicadores de status são padrões consolidados nesse tipo de interface.

3 Metodologia

O sistema foi desenvolvido em três camadas principais, conforme descrito a seguir.

3.1 Backend — API RESTful com Node.js

A camada de backend foi desenvolvida utilizando Node.js com o framework Express. A API opera na porta 8080 e utiliza armazenamento em memória para os dados dos sensores. Os endpoints implementados estão descritos na Tabela 1.

Tabela 1: Endpoints da API RESTful.

Método	Rota	Descrição
GET	/iot	Retorna todos os dados de sensores
GET	/sensor/:id	Retorna um sensor pelo ID
GET	/usuarios	Retorna a lista de usuários
GET	/devices	Retorna a lista de dispositivos
POST	/newData	Cria um novo registro de sensor
PUT	/sensor/:id	Atualiza ou cria um sensor (upsert)

O endpoint `PUT /sensor/:id` foi implementado com lógica de *upsert*: caso o sensor com o ID informado já exista, seus dados são atualizados; caso contrário, um novo registro é inserido. Isso elimina a necessidade de criação prévia dos sensores antes de iniciar a atualização.

3.2 Simulação de Sensores — Node-RED

O Node-RED foi utilizado para simular o comportamento de dispositivos IoT reais. O fluxo implementado é composto por dois subfluxos:

- **Criação de sensor:** um nó *Inject* dispara manualmente uma requisição `POST /newData` com valores iniciais zerados, criando o primeiro registro na API.
- **Atualização periódica:** um nó *Inject* configurado para disparar a cada 5 segundos aciona uma função que gera dados aleatórios para os sensores de ID 1, 2 e 3. Para cada sensor, a função monta uma mensagem com a URL dinâmica `http://localhost:8080/sensor/{id}` e envia uma requisição `PUT` com os novos valores.

A geração de dados randômicos na função Node-RED segue a seguinte lógica para cada sensor:

- Temperatura: valor aleatório entre 0 e 100 °C
- Pressão: valor aleatório entre 900 e 1100 hPa
- Umidade: valor aleatório entre 0 e 100%
- Sensor de presença e trava de segurança: valores booleanos aleatórios

3.3 Frontend — Dashboard HMI com Next.js

A interface de usuário foi desenvolvida com Next.js 16 e React 19, utilizando Tailwind CSS 4 para estilização. O dashboard apresenta tema escuro (*dark mode*) e visual industrial, características típicas de sistemas HMI.

A atualização dos dados ocorre automaticamente a cada 3 segundos por meio de um intervalo configurado com o hook `useEffect` do React, que realiza requisições `GET /iot` à API.

A interface é composta por dois elementos principais:

- **Painéis de sensores:** três painéis exibidos lado a lado, um para cada sensor. Cada painel contém gauges circulares SVG para temperatura, pressão e umidade, além de indicadores LED para o sensor de presença e a trava de segurança.
- **Tabela histórica:** exibe todas as leituras registradas na API, com as leituras mais recentes no topo, e permite editar individualmente cada registro via modal.

Os gauges circulares foram implementados diretamente em SVG, utilizando a propriedade `stroke-dasharray` para criar o arco proporcional ao valor atual dentro do intervalo mínimo-máximo de cada grandeza. Um efeito de *glow* é aplicado ao arco quando o valor é maior que zero.

4 Resultados e Discussões

O sistema apresentou os seguintes resultados:

- Monitoramento em tempo real dos três sensores com atualização a cada 3 segundos no frontend e envio de novos dados a cada 5 segundos pelo Node-RED.
- Visualização clara dos valores de temperatura, pressão e umidade por meio de gauges circulares com animação suave na transição entre leituras.
- Indicadores LED de presença e trava de segurança com efeito de brilho verde quando ativos.
- Histórico completo de todas as leituras com possibilidade de edição via modal.
- Topbar com indicador de status da conexão com a API (ONLINE/DESCONECTADO) e horário da última sincronização.

A Tabela 2 exemplifica os dados monitorados pelo sistema.

Tabela 2: Exemplo de leitura dos sensores monitorados.

Sensor	Valor	Unidade
Temperatura	45,3	°C
Pressão	1013	hPa
Umidade	62	%
Presença	Detectada	—
Trava	Ativada	—

A arquitetura adotada mostrou-se eficiente para o cenário proposto. A separação entre backend, simulador e frontend facilita a manutenção e a escalabilidade do sistema. A utilização do Node-RED como camada de simulação permite substituí-lo futuramente por dispositivos físicos reais sem alterações no backend ou no frontend.

5 Conclusão

O projeto demonstrou a viabilidade de integrar sensores e atuadores em uma plataforma IoT com API RESTful, simulação via Node-RED e interface HMI desenvolvida com tecnologias web modernas. A solução é de baixo custo de implementação e pode ser adaptada para diferentes cenários de automação.

Como trabalhos futuros, pretende-se: (i) substituir o armazenamento em memória por um banco de dados persistente; (ii) adicionar autenticação aos endpoints da API; (iii) integrar dispositivos físicos reais ao sistema, substituindo a simulação do Node-RED; e (iv) incorporar análise preditiva com base no histórico de leituras.

Agradecimentos

Agradeço à Faculdade de Tecnologia SENAI Gaspar Ricardo Júnior e ao orientador Gabriel, que contribuiu para o desenvolvimento deste trabalho, e aos colegas da Turma Toyota-3 pelo apoio durante o processo de aprendizagem.

Referências

IBM. **Node-RED: Low-code programming for event-driven applications**. 2023. Disponível em: <https://nodered.org>. Acesso em: 2025.